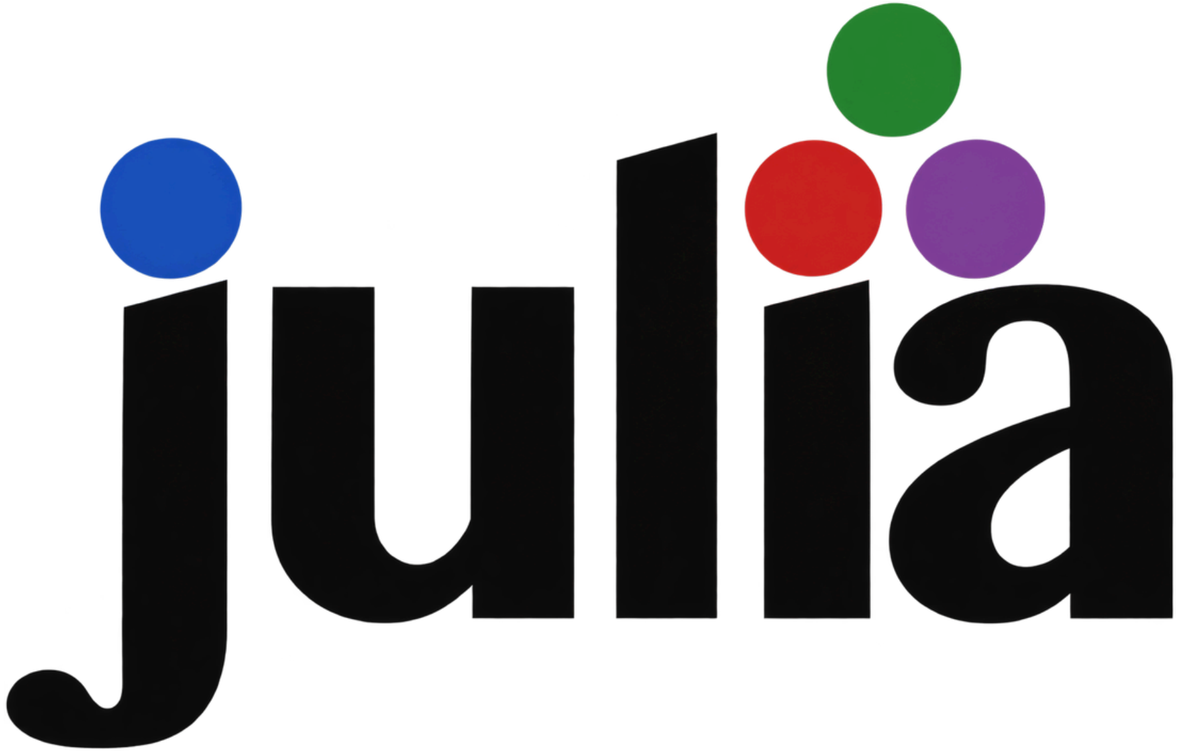


Python-like ease. C-like speed.

Ana X. Garcia Suarez

Josue Torres

Neale Matthew Nicolas

Gael Rodriguez

Fun fact!

The Julia Programming Language is named the way it is for no particular reason.



Sources:
https://leftronic.com/blog/julia-programming-language
https://www.infoworld.com/article/2285732/new-julia-language-seeks-to-be-the-c-for-scientists-2.html

Outline

1. **Introduction to Julia**
   What it is and why it exists.
2. **Syntax & Control Structures**
   Loops, conditionals, and branching.
3. **Functions & Subprograms**
   Standard, compact, and generic methods.
4. **Key Innovations**
   Multiple Dispatch and Metaprogramming.
5. **Main Applications**
   Where Julia is used in the real world.
6. **Summary & Review**
   Key takeaways and questions.

What is Julia?

A high-level, dynamic programming language designed specifically for high-performance numerical and scientific computing.

Python / R

Easy to write

Slower to run

+

C / Fortran

Fast to run

Harder to write

=

Julia

Easy to write

Fast to run

Core Purpose: solve the “Two-Language Problem” in software engineering

Prototype in an easy language, then rewrite in a fast one.

Brief History

2009

Started at MIT

2012

Publicly announced

2018

Version 1.0 released

Creators

Jeff Bezanson, Stefan Karpinski, Viral B. Shah, and Alan Edelman

Language Category

PARADIGM

Multi-paradigm

Procedural, step-by-step code and functional programming with math-like functions.

CORE ARCHITECTURE

Multiple dispatch

Selects a method using the types of all input arguments.

TYPING DISCIPLINE

Dynamically typed

Variables need no type declaration.
Type annotations are optional.

EXECUTION MODEL

Just-in-time compiled

Compiles specialized methods into fast native machine code.

Multiple dispatch in action

combine(a, b) can select different methods for two numbers or two matrices.

Versions & Dialects

FOUNDATION · 2018

1.0

Language stability

A commitment to backward compatibility across Julia 1.x. Existing language code keeps working as Julia evolves.

CURRENT STABLE

1.10+

Faster first runs

Native-code caching since 1.9 greatly reduces warm-up time. Initial compilation can still cause a delay.

LONG-TERM SUPPORT

1.10.12

Careful upgrades

A maintained release branch for stability-focused projects. Receives maintenance fixes while supported.

One official language, a shared package ecosystem

Julia has no major competing dialects. Packages share the same language and tools. Package version requirements still matter for compatibility.

Programming Environments

The REPL

Julia's built-in interactive prompt.
Run code directly in the terminal.

Press] for package mode julia>]

pkg> add Plots

Press ? for help mode julia> ?

help?> sum

At an empty prompt, press Backspace to go back a path. Backspace returns to julia>.

VS Code

A project editor with the Julia extension.
Integrated REPL, debugging, and a plot pane.

Pluto.jl

Reactive notebooks: change an input and all dependent cells update automatically.

Jupyter + IJulia

Classic notebooks for code, text, and results.
Rerun dependent cells to refresh their output.

Why "Jupyter"? Its name combines **Julia**, **Python**, and **R**.

Pros & Cons

Pros

High performance, readable code

Can approach C/C++ speed for numerical work with the convenience of high-level syntax.

Math-friendly syntax

Natural formulas, array operations, and Unicode symbols make math easier to express.

```
α = 0.5    y = α * x + β
```

Cons

Initial compilation delay

The JIT compiler can add a pause before new code runs for the first time.

Smaller ecosystem

More specialized than Python, with fewer third-party libraries and job opportunities.

Indexing adjustment

Standard arrays start at 1 instead of 0.

> **The first item in [10, 20, 30]**
>
> Python / C++ **a[0]** = 10 Julia **a[1]** = 10

Real-World Applications

Scientific machine learning

Combines physics-based models with ML to study systems such as fluid flow and heat transfer.

SciML: differential equations + learning

Aerospace & climate

Develops aircraft collision-avoidance logic and simulates Earth's atmosphere and climate.

FAA / MIT Lincoln Laboratory · CliMA

Computational finance

Runs quantitative investment models and repeated financial simulations to support investment decisions.

Betterment: projections and recommendations

Pharmaceuticals

Simulates how therapies affect patients and virtual populations to support drug-development research.

Pfizer: quantitative systems pharmacology

Evaluation — Readability

Mathematical syntax

Numeric coefficients keep expressions close to familiar mathematical notation.

```
# 2x means 2 * x
x = 2
y = 2x + 1
5
```

Native Unicode support

Greek letters and many mathematical symbols work directly in variable names.

```
α = 0.5
β = 2.0
```

Explicit block endings

The end keyword closes code blocks. Indentation helps readers see the structure.

```
if x > 0
println(x)
end
```

Consistent operators across types

Familiar notation works for supported types. The meaning still follows the data type.

```
a + b # numbers
A + B # compatible matrices
```

Readability caveat: A * B is matrix multiplication; A .* B multiplies elements individually.

Evaluation — Writability

Dynamic typing

Assign values without declaring a variable's type upfront.

```
x = 5
```

Less setup for quick experiments.

Multiple dispatch

One function name, methods for different argument types.

```
combine(a, b)
```

Julia selects the matching method. Generic methods can share logic.

High expressivity

Matrix multiplication in one line.

```
A = [1 2; 3 4]
B = [5 6; 7 8]
C = A * B
```

```
19 22
43 50
```

Handwritten C++: nested i, j, k loops. C++ libraries can also be concise.

Metaprogramming: macros automate repeated code

A timing example: wrap a function call with @time.

```
# Manual timing concept
start = time_ns()
result = my_function()
elapsed = (time_ns() - start) / 1e9
println(elapsed)
```

```
@time my_function()
```

Runs the function, prints timing and memory allocation details, and returns its result.

The first timing can include compilation. For repeated benchmarks, use BenchmarkTools.jl.

Evaluation — Reliability

Automatic memory



Garbage collection reclaims unused managed objects.

✓ Reduces manual memory errors

Optional Type Annotations



Add strict types whenever needed for clarity and performance.

✓ Catches errors before execution, unlike standard Python

Immutable data structures



Provides built-in protection for variables against unwanted side effects.

✓ Makes multi-threaded & parallel code much safer

Clear error handling



Clear exceptions expose failures. try/catch supports recovery.

✓ Makes failures easier to handle

Built-in safeguards reduce common programming errors.
Reliable results still need testing, numerical checks, and safe handling of shared data.

Evaluation — Cost

Software	**No license fee**	Julia is free and open source. It can replace MATLAB for suitable workflows, depending on required toolboxes.
Training	**Some retraining**	Familiar syntax helps Python and C++ developers. Learn multiple dispatch, type patterns, and 1-based arrays.
Development	**High Potential savings**	Prototype and production code can share one language. Less rewriting can save time, with no fixed reduction guaranteed.
Execution	**Highly Efficient**	Fast numerical code can reduce compute time and cloud cost. Measure the full workload, including startup and compilation.
Deployment	**Moderate Costs**	JIT compiler. Must use PackageCompiler tool which can bundle an app with Julia runtime libraries. Builds and bundle size need more planning than a small C++ binary.

No language license fee. Total cost depends on training, workload, and deployment needs.

Syntax

Newlines end statements

No semicolon needed after a complete expression.

```
x = 5
y = x + 1
```

Blocks close with end

Indent for readability; end defines the boundary.

```
if x > 0
println(x)
end
```

Numeric coefficients

A number beside a variable implies multiplication.

```
y = 2x + 1
# Same as y = 2 * x + 1
```

Arrays start at 1

The first element of a standard array has index 1.

```
a = [10, 20, 30]
a[1] # returns 10
```

Two useful details: semicolons can suppress REPL output, and x * y still needs the multiplication sign.

Julia Keywords and Syntax

Modules

```
module
export
public
import
using
as
baremodule
```

Functions and blocks

```
function
macro
return
do
begin
end
let
```

Control flow

```
if
for
while
break
continue
try
finally
```

Scope and types

```
local   global   outer
const
struct
mutable struct
abstract type
primitive type
where
```

Quoting and expressions

```
quote
...
;
=
?:
```

Operators and syntax

```
.=
.
->
::
[]
```

Special Characters & Block Delimiters

end	**Block endings**
	Keywords open blocks; end closes them.
	Indentation helps readability.
		`if x > 0`
		`println(x)`
		`end`

!	**The Bang Symbol** A trailing ! signals that a function modifies one or more arguments.	`sort!(x)` # sorts x in place
.+	**Broadcasting** Dotted operators act element by element. For functions, use the same idea: sin.(x).	`[1, 2, 3] .+ 10` # gives [11, 12, 13]
::	**Type annotations** Declares a type or checks a value's type. A failed type assertion raises an error.	`value::Int64` # asserts an Int64 value

Remember: ! communicates intent; . applies an operation across elements; :: expresses a type constraint.

Data Types

Primitive numeric types

Fixed-size representations for numbers and booleans.
Values have types; variables can be dynamic.

```
42 # Int64 on 64-bit systems
3.14 # Float64
true # Bool
```

Strings vs. characters

Single quotes identify one character.
Double quotes create a string.

```
'a' # Char
"abc" # String
"a" # also a String
```

Arrays & matrices

Typed, N-dimensional containers for data and math.
Vectors are 1D; matrices are 2D.

```
v = [1, 2, 3] # vector
A = [1 2; 3 4] # matrix
# spaces: columns; semicolons: rows
```

Structs: custom types

Group related fields into an immutable type.
Use mutable struct when fields must change.

```
struct Point
x::Float64
y::Float64
end
```

Julia defines functions separately from structs and selects methods using multiple dispatch.

Scope

Lexical (static) scoping

Visibility follows where code is defined,
rather than where a function is called.

Global vs. Local

Modules: define global scopes.
Functions: let blocks, and loops create local scopes.

Global: reassign an outer name

Inside a function, use global to reassign
a module-global variable. Reading it needs no keyword.

Local: make a separate binding

Explicitly creates a variable in a local scope.
It can shadow an outer variable with the same name.

One global variable, two local scopes

```
x = 10
function update()
global x = 20
local y = 5
return y
end
update()
x # 20
let
local x = 99
end
x # still 20
```

Scope detail: files alone create no scope; global-level loops have soft-scope rules that differ in scripts and the REPL.

Expressions & Operators

Blocks can return values

An if expression returns its chosen branch's value.
Assign that result directly to a variable.

```
x = if “condition”
5
else 10 end
```

Dotted operators

Apply an operation element by element.
Compatible arrays and scalars can broadcast.

```
[1, 2, 3] .+ [4, 5, 6]
# gives [5, 7, 9]
```

Exact rational numbers

Use // to preserve a fraction exactly.
A single / performs ordinary division.

```
1//3 # exact rational
1/3 # 0.3333333333333333
```

Chained comparisons

Express a range check in mathematical notation.
No need to repeat the middle operand.

```
1 < x < 10
# true when x is between 1 and 10
```

Two distinctions: loops normally return nothing, and broadcasting does not automatically mean parallel execution.

Operator Precedence and Associativity

Highest precedence at the top, lowest at the bottom

Category	Operators	Associativity
Syntax	`.` followed by `::`	Left
Exponentiation	`^`	Right
Unary	`+` `-` `!` `~` `¬` `√` `∛` `∜` `★` `±` `∓` `<:` `>:`	Right
Bitshifts	`<<` `>>` `>>>`	Left
Fractions	`//`	Left
Multiplication	`*` `/` `%` `&` `\` `÷`	Left
Addition	`+` `-` `|` `V`	Left
Syntax	`:` `..`	Left
Syntax	`|>`	Left
Syntax	`<|`	Right
Comparisons	`>` `<` `>=` `<=` `==` `===` `!=` `!==` `<:`	Non-associative
Control flow	`&&` followed by `||` followed by `?`	Right
Pair	`=>`	Right
Assignments	`=` `+=` `-=` `*=` `/=` `//=` `\=` `^=` `÷=` `%=` `|=` `&=` `V=` `<<=` `>>=` `>>>=`	Right

Control Structures

Loops: for & while

Ranges make iteration concise.
Both loop forms close with end.

```
for i in 1:3
println(i)
end
```

Compound expressions

begin groups expressions without a new scope.
The block returns its last expression’s value.

```
total = begin
a = 2; b = 3
a + b
end # total is 5
```

Short-circuit logic

&& skips the right side if the left side is false.
|| skips it if the left side is true.

```
isValid && saveFile()
isReady || showWarning()
```

Exception handling

catch handles a thrown exception.
Recover, report it, or rethrow when appropriate.

```
try
parse(Int, "oops")
catch err
println("Invalid number")
end
```

Use loops to repeat, begin to group, short-circuit logic to skip, and try/catch to handle exceptions.

Selection & Repetition

if · elseif · else

elseif is one word. No colon needed.
The conditional block closes with end.

```
if x > 0
println("Pos")
elseif x < 0
println("Neg")
else
println("Zero")
end
```

Ternary: a one-line decision

Use condition ? true_value : false_value.

```
status = score > 60 ? "Pass" : "Fail"
```

Combined loop headers

Comma-separated iterators express nested loops.

```
for i in 1:3, j in 1:3
println(i * j)
end
```

Array comprehensions

Build an array and filter values in one expression.

```
evens = [x for x in 1:10 if x % 2 == 0]
# [2, 4, 6, 8, 10]
```

Ternary syntax needs spaces around ? and :. Combined loop headers still run nested iterations.

Unconditional Branching

break: leave the loop

Exits the innermost loop immediately.

```
for i in 1:10
i == 5 && break
println(i)
end # prints 1, 2, 3, 4
```

continue: skip this iteration

Skips the remaining body and starts the next cycle.

```
for i in 1:5
iseven(i) && continue
println(i)
end # prints 1, 3, 5
```

return: leave the function

Ends the function and returns the supplied value.

```
function check(x)
x < 0 && return false
return true
end
```

@goto: jump to a label

Transfers control to a matching @label.

```
function countup()
x = 0
@label again
x += 1
x < 5 && @goto again
return x
end
```

A condition can decide when to jump. Prefer structured loops and functions over @goto in ordinary code.

Functions & Subprograms

Standard functions

Use function…end for a reusable block of code.
Arguments appear in parentheses.

```
function add(a, b)
return a + b
end
add(2, 3) # 5
```

Compact functions

Define a short function with an equation.
The function keyword is optional in this form.

```
square(x) = x * x
square(4) # 16
```

Anonymous functions

Use -> for a function without a name.
Useful when passing a small operation to a function.

```
x -> x + 1
map(x -> x + 1, [1, 2, 3])
# [2, 3, 4]
```

Implicit returns

The last evaluated expression supplies the result.
An explicit return can exit the function earlier.

```
function get_five()
5
end
get_five() # 5
```

Returning a value is different from printing it. Use println when you want visible output.

Methods & Parameters

Methods & multiple dispatch

The argument types select the matching method.
Dispatch considers all positional arguments.

```
say_hi(x::Int) = "A number!"
say_hi(x::String) = "A word!"
say_hi(5) # "A number!"
```

Keyword arguments

A semicolon separates named keyword arguments.
Defaults make them optional.

```
greet(name; phrase="Hello") =
"$phrase, $name!"
greet("Bob"; phrase="Yo!")
# "Yo!, Bob!"
```

Multiple return values

Return a tuple, then unpack its elements
into separate variables.

```
pair(a, b) = (a, b)
first, second = pair(2, 3)
# first is 2; second is 3
```

Pass-by-sharing

Arguments refer to the same values as the caller.
Mutating an array changes that shared object.

```
add_five!(v) = push!(v, 5)
items = [1, 2]
add_five!(items)
# items is now [1, 2, 5]
```

Shared data can be mutated; reassigning a parameter name does not reassign the caller's variable.

Generic Subprograms

Generic by default

Omit argument types to accept any values.
The operations inside must support them.

```
add(x, y) = x + y
add(2, 3) # 5
add([1, 2], [3, 4]) # [4, 6]
```

Parametric methods

where T names a type parameter.
Here, both inputs need the same concrete type.

```
compare(x::T, y::T) where T = x == y
compare(2, 3) # false
compare(2, 2.0) # MethodError
```

Type constraints

T <: Number limits T to numeric types.
A family of types can share one method.

```
do_math(x::T) where {T <: Number} = x + 10
do_math(2.5) # 12.5
do_math("two") # MethodError
```

Start generic, then add type relationships or constraints when the algorithm needs them.

Two Language Innovations

Multiple dispatch

Functions adapt to all positional inputs

Julia selects the matching method from the types of every positional argument, not just one object.

```
struct Asteroid end
struct Ship end
struct Planet end

collide(a::Asteroid, b::Ship) =
    "Ship takes damage!"
collide(a::Asteroid, b::Planet) =
    "Crater formed!"

collide(Asteroid(), Ship())
```

Result: "Ship takes damage!"

Metaprogramming

Macros transform code before execution

Invoke a macro with @ to wrap an expression or introduce specialized syntax.

```
function work()
    total = Int64(0)
    for i in 1:1_000_000
        total += 2 * i
    end
    return total
end

@time work()
# reports time and allocations
```

The macro adds measurement around the call.

The first @time measurement can include compilation. Use BenchmarkTools.jl for repeated performance comparisons.

Main Applications

Scientific computing & simulation

Model physical systems

Simulate dynamics, climate, and aerospace systems.
Reuse numerical models from research to production.

Tools: DifferentialEquations.jl · SciML

Machine learning & AI

Build and train models

Create neural networks and differentiable programs.
Combine learning with Julia’s numerical tools.

Tools: Flux.jl · automatic differentiation

Data science & analytics

Transform data into insights

Clean, join, and summarize tabular datasets.
Use optimized loops and threading where appropriate.

Tools: DataFrames.jl · JuliaData ecosystem

Quantitative finance

Evaluate decisions and risk

Run investment projections, pricing, and risk models.
Iterate rapidly on computational strategies.

Example: Betterment’s investment modeling

A shared strength: mathematical modeling, fast computation, and rapid experimentation in one language.

Summary

Julia brings mathematical ideas closer to fast, working code.

High-level code. Native performance.

Readable syntax with compiled numerical performance that can approach C/C++ on suitable workloads.

Math-first design

Strong tools for scientific computing, data analysis, machine learning, and numerical simulation.

Two defining capabilities

Multiple dispatch selects methods by argument types.
Macros transform code before execution.

Convenient everyday development

Optional line-ending semicolons, explicit end blocks, and automatic management of ordinary Julia memory.

> **The main tradeoff: deployment**
>
> Small, standalone executables usually take more packaging work than in C++.
> Plan for runtime dependencies and bundle size.

Review Questions

1

TRUE OR FALSE?

Julia requires manual memory management, as in C++, instead of automatic garbage collection.

2

MULTIPLE CHOICE

What primary field was Julia designed for?

A) Front-end web development

B) Scientific computing & data science

C) Mobile app creation

D) Operating systems

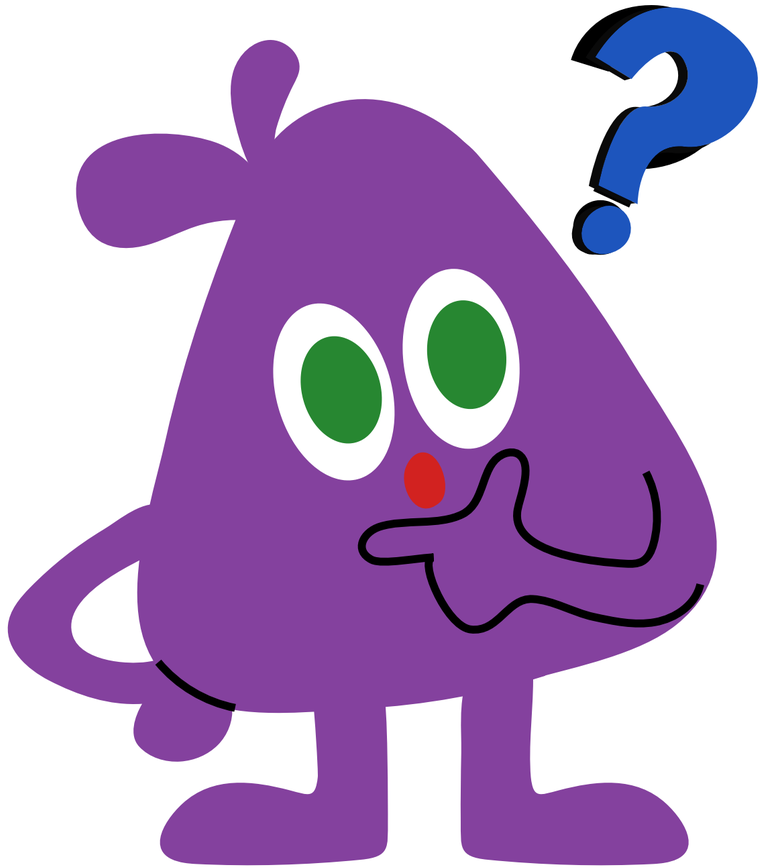
3

TRUE OR FALSE?

Julia can address the “two-language problem”: prototype and get high performance in one language, without a mandatory C++ rewrite.

Final Review Question

What is the reason Julia is named the way it is?



References

OFFICIAL WEBSITE

The Julia Project. (n.d.). *The Julia Programming Language.*
https://julialang.org/

OFFICIAL DOCUMENTATION

The Julia Project. (n.d.). *The Julia Language Documentation.*
https://docs.julialang.org/

RESEARCH PAPER

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017).
Julia: A fresh approach to numerical computing. *SIAM Review, 59*(1), 65–98.
https://doi.org/10.1137/141000671

TEXTBOOK

Lauwens, B., & Downey, A. B. (2019).
Think Julia: How to think like a computer scientist. O'Reilly Media.

https://leftronic.com/blog/julia-programming-language

https://www.infoworld.com/article/2285732/new-julia-language-seeks-to-be-the-c-for-scientists-2.html

THANK YOU

